

Coding & Solution

Date	01 July 2026
Team ID	
Project Name	Rising Waters: Smart AI Flood Prediction System
Maximum Marks	5 Marks

Coding & Solution

This section evaluates the quality and completeness of your implemented code and the overall solution. Your submission should demonstrate working functionality, clean code practices, and alignment with your defined requirements.

Instructions:

1. Ensure your code is well-structured, commented, and follows best practices for your chosen technology stack.
2. The solution should address the core problem statement and implement the key functional requirements.
3. Include all source files, configuration files, and setup instructions needed to run the application.
4. Attach or link to your code repository (e.g., GitHub) in the table below.

Solution Summary

Field	Details
Repository Link / URL	[Paste your GitHub / GitLab or hosted link here]
Programming Language(s)	Python, HTML5, CSS3, JavaScript
Framework(s) Used	Flask (Web Framework), PyTorch, Scikit-Learn, XGBoost
Key Features Implemented	* Multi-Model Predictive Core: Implemented and cross-validated Decision Tree, Random Forest, KNN, and XGBoost models. * High-Accuracy Engine: Selected optimized XGBoost classifier achieving 96.55% test accuracy. * Robust Preprocessing Pipeline: Standardized feature pipelines to handle annual rainfall data, cloud visibility, and seasonal variations. * Interactive Flask Dashboard: Modern web interface for

	inputting current meteorological features with instantaneous flood chance categorization. * Cloud-Ready Deployment: Scalable application architecture structured for zero-downtime hosting on IBM Cloud.
Pending / Incomplete Features	* Integration of automated real-time weather API webhooks to dynamically fetch live telemetry data instead of manual entry. * Historical flood data geolocation map layer overlay integrated inside the user interface dashboard.
Setup / Run Instructions	1. Environment Configuration: conda create -n flood_pred python=3.9 conda activate flood_pred 2. Dependency Installation: pip install flask numpy pandas scikit-learn xgboost torch matplotlib seaborn joblib 3. Application Runtime Execution: Verify the pre-compiled serialization file floods.save is placed in the model folder, and run: python app.py 4. Application Access: Open a modern web browser and navigate to: http://127.0.0.1:5000/

Code Quality Checklist

S.No	Criteria	Status (Yes / No)
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Yes
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Additional Notes / Comments:

The flood prediction system codebase meets production-level quality standards. The data preprocessing steps, model inference pipeline, and Flask web application routing are completely decoupled to ensure modularity. Comprehensive Exception Handling blocks (try-except) are integrated within the Flask API to manage missing user payloads or anomalous inputs without system crashes. The code repository is properly initialized with a clear .gitignore file to ensure security and prevent the upload of unnecessary temporary checkpoints or execution environment cache files.